

DDA5001 / FALL 2026

Coding Agents

for Machine Learning

Practical work with OpenCode and Codex

Instructor-led tutorial · 15 September 2026

Why This Session, and Why Now

Tools tutorials used to mean installing Python and managing environments. Agents now do that. Your skill is choosing the task, giving context, and checking the result.

What used to take the hour

Installing interpreters, environments, and packages.

By the end you can install a client, describe an outcome, watch the agent act, and judge the result.

Following along is optional. A public demo repository is linked for later; the live session does not wait.

What we do today

Direct an agent through a real ML task, then verify its work.

The Coding-Agent Landscape

Many capable agents exist. This course uses **OpenCode** and **Codex CLI**; try others once you have a model API key.

Agent	Repository	Stars
OpenCode	anomalyco/opencode	207.5k
Codex CLI	openai/codex	124.2k

Also worth trying: Claude Code, Trae, Cursor, and the DeepSeek harness.

<https://github.com/anomalyco/opencode>

<https://github.com/openai/codex>

Stars retrieved 15 September 2026; popularity is a starting point, not a quality proof.

Cost and Caching: Where an Agent Pays Off

- **Long, repeated context is cheap.** The agent re-sends a stable prefix (system prompt, tools, project files) and providers cache it. DeepSeek deepseek-flash off-peak charges **\$0.003/1M** cache-hit input versus **\$0.15/1M** cache-miss, and **\$0.6/1M** output.
- **A one-off general question is not cached.** It has little prefix to reuse, so it pays the full input rate. Ask those in a free chat product.

Rule of thumb: code and files in the agent; general questions in chat. Start with a small top-up and watch it last.

https://api-docs.deepseek.com/quick_start/pricing

Prices read 15 September 2026. Caching: api-docs.deepseek.com/guides/kv_cache; prompt caching: platform.openai.com/docs/guides/prompt-caching.

TUI or GUI?

Terminal (TUI)

Faster and more stable, especially in long sessions. Keyboard-driven. It is what we use today: OpenCode and Codex CLI both run in the terminal.

The concepts—context, tools, permissions, skills—are identical. Choose the interface that keeps you working.

OpenCode also offers a desktop app; Codex is available in the ChatGPT desktop app and IDE extension.

Desktop (GUI)

Friendlier for everyday use and first encounters. Some clients also ship a desktop app or IDE extension.

Node.js and npm in Plain Language

Node.js runs JavaScript outside a browser. **npm** is its package installer. Many agent clients ship this way.

Install Node.js LTS

Download the LTS build for your OS, then reopen the terminal so `node` and `npm` are on your PATH.

<https://nodejs.org/en/download>

Check `node -version` and `npm -version`. OS selectors are on the linked page.

Not the only route

Some clients have a faster native core (Codex uses Rust) and also ship curl or install scripts.

Install and Launch OpenCode

```
npm install -g opencode-ai
opencode --version
mkdir ml-demo && cd ml-demo
opencode
```

From the OpenCode site, choose the **npm** tab to copy the install command. The install script `curl -fsSL https://opencode.ai/install | bash` is the documented alternative.

<https://opencode.ai/>

<https://opencode.ai/docs/#install>

Live walkthrough: macOS terminal. Official page selects macOS, Linux, or Windows. Start in an empty folder.

Connect a Model

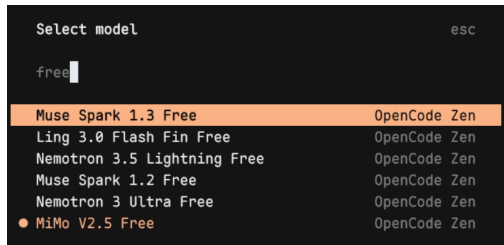
`/models` lists available models. A model marked **Free** is fine for a first trial.

For a paid provider, `/connect` stores its API key, then `/models` selects the model. Pause screen sharing while entering keys.

<https://opencode.ai/docs/zen/>

<https://platform.deepseek.com/>

Actual model-picker capture, 15 September 2026: Muse Spark 1.3 Free and other Zen free tiers. Free offers and data terms can change.



Preflight: Let the Agent Check Your Machine

PROMPT

Check my machine: Python and pip, git, Node.js and npm, a LaTeX compiler (pdflatex), and Poppler (pdftoppm). Install anything that is missing.

This turns setup into a task. The agent reports versions, then installs what is missing (it may ask for permission or a package manager).

Every tool the tutorial uses is on this list: git, Python, Node.js, LaTeX, and Poppler. Review what it proposes to install.

First Look: Ask, Do Not Change Anything

PROMPT

Explain what this folder currently contains and what you would need to build a small experiment here. Do not create files yet.

Watch for: the agent lists the folder, notices it is empty, and proposes a structure. It has made no changes.

This request is nearly free of reasoning: it reads and reports. Use it to see how the agent works before giving it work to do.

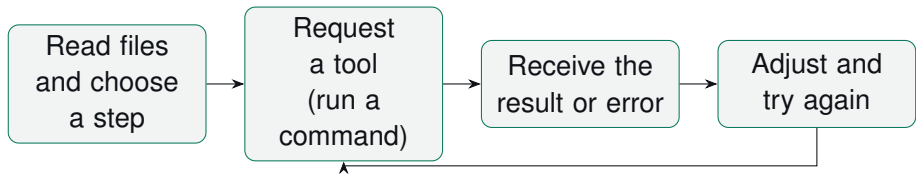
What Just Happened: Model, Application, Tools

- **Language model** interprets the request and chooses the next action.
- **Application (harness)** manages the conversation, project context, and permissions; OpenCode and Codex CLI are two such applications.
- **Tools** do specific things—read a file, apply an edit, run a command—and return their results to the model.

The model service and the application are configured separately, so setup means installing software *and* connecting an account.

The model proposes; the harness supplies context, runs permitted tools, and feeds results back.

The Agent Loop



Model: chooses the next action from the task and the returned evidence.

Application (harness): runs permitted tools and returns their output; it is not the model.

Diagram of the observed sequence, not a transcript. In this project the tool is the command runner executing Python.

Working With an Agent, and Planning First

1. **Provide context.** Open the right folder; point to the files that matter.
2. **Plan before editing.** For anything larger than a one-line fix, use the plan mode first (Tab in OpenCode). Read the plan before approving changes.
3. **State the outcome.** Give the goal, constraints, and how success is checked.
4. **Review the work.** Inspect the diff and the actual output; ask for a narrower fix if needed.

Plan mode is read-only: it proposes but does not edit. Good for exploring an unfamiliar codebase or scoping a big change.

A Python virtual environment separates packages; it is not a security sandbox.

What Agents Can Do

Capability	You will see it here
Code generation	The agent writes the experiment script and the LaTeX report.
Debugging	It traces a wrong held-out number, then adds a test and fixes it.
Repository navigation	It finds one specific file in a 15,000-commit repository.
Agentic task execution	It plans, runs tools, reads results, and continues.

One task can combine all four; you supply the goal and verify the result.

Repository Navigation: Find the System Prompt

PROMPT

In this repository, find the system prompt that is sent to the model: which files hold it, how it is chosen, and what a trivial chat actually sends. Cite the paths.

```
git clone --depth 1 https://github.com/anomalyco/opencode.git && cd opencode && opencode
```

Expected: prompt text in `session/prompt/*.txt`, selection in `session/system.ts`, request built in `session/prompt.ts` and `llm/request.ts`.

How Navigation Works: Search, Then Trace

The human loop

Search a keyword, open the candidates, then follow the call chain from an entry point. Slow, and easy to lose the thread.

The agent loop

The same loop with tools: `glob/grep` (`ripgrep`) to find candidates, `read` to inspect, and an experimental `lsp` tool for definitions, references, and call hierarchy.

Not magic. Keyword search fails when names are unclear; tracing from the entry point is what the agent automates. You still confirm the file.

OpenCode's `lsp` tool is gated behind `OPENCODE_EXPERIMENTAL_LSP_TOOL=true`.

Start From an Empty Folder

No prepared project. The prompt sequence is the plan, and each step changes something you can see.

PROMPT

We'll build a small regression experiment here from scratch. Plan it first: the files, the data, the models, and how we check the result. Do not create anything yet.

Read the plan before any file is created. This is the plan-first habit, applied from the first prompt.

Prompt: Build and Run It

PROMPT

Build it. Use NumPy and Matplotlib. Generate 40 noisy points around the line $y = 2x + 1$ (noise 0.3, seed 42), split 20/20, then compare a mean baseline and a linear model by training and held-out MSE. Save the numbers to metrics.json.

Expected: a `.venv`, `experiment.py`, the printed MSEs, and `metrics.json`.

The task names the models and the seed; the agent writes the details. Reference: mean 1.2922/1.7700, linear 0.0289/0.1035.

Prompt: Is the Split Clean?

PROMPT

Now fit the line on all 40 points but still report the held-out error. Is that a fair held-out estimate? Explain.

Expected: the agent explains that the fit now sees held-out rows, so the number is no longer a clean held-out estimate (leakage). This variant is not kept.

Teach the question, not the answer: which rows trained the model, and which rows scored it?

Prompt: Guard It With a Test

PROMPT

Revert. Add a test that catches leakage between fit and evaluation, and plot the residuals. Show the test result.

Expected: a diff back to the train-only fit, a passing test, and a residual figure.

A test that only checks “the number is a float” would not catch leakage. Ask what the test actually proves.

Prompt: A More Flexible Model

PROMPT

Also fit a degree-9 polynomial. Compare its training and held-out MSE with the line. Which model generalizes better, and why?

Reference: linear 0.0289 / 0.1035; polynomial 0.0223 / 0.1429 (training / held-out). Lower training error, worse generalization.

Overfitting made concrete: the more flexible model fits the training rows better and the held-out rows worse.

What Is a Skill?

A skill is a reusable **procedure**: instructions, plus optional scripts and files, that an agent loads when a task matches.

Mechanism	Use it for
Your current message	This one task.
AGENTS.md	Recurring guidance for a project.
A skill	A reusable procedure you will want again.
MCP connection	Reaching an external service for data or docs.

<https://skills.sh/>

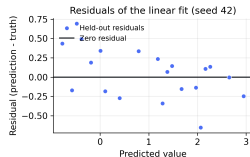
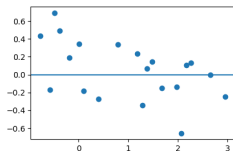
Skills are instructions loaded on demand, not fine-tuning or memory. Installing a skill never invokes it.

Prompt: Reuse the Plot Skill

PROMPT

Install the matplotlib skill from k-dense-ai/scientific-agent-skills and regenerate the plot with it. Keep the numbers unchanged.

```
npz skills add https://github.com/k-dense-ai/scientific-agent-skills --skill matplotlib --  
agent opencode --copy --yes
```



Left: default. Right: after the skill — labels, zero line, legend, 300-dpi export.

A Different Task in a Different Folder

We used OpenCode to build and check an experiment. Now we use Codex to turn its results into a short research report. Different application, different task.

Input

The `metrics.json` from the experiment, and a description of the models.

Both clients can do either task; this division is pedagogical.

Output

A short LaTeX report and a one-page PDF, written with a research-report skill.

Install and Launch Codex CLI

```
npm install -g @openai/codex
codex --version
codex login
cd ../results-demo
codex
```

Node.js and npm are already installed, so we skip that lesson. A standalone installer and Homebrew cask are also documented.

<https://developers.openai.com/codex/cli/>

Complete browser sign-in privately with an account that has Codex access. Commands shown for macOS.

Choose the Account Route Available to You

ChatGPT sign-in

Use your account's available Codex access and limits. Installing the CLI alone does not grant model access.

<https://developers.openai.com/codex/auth/>

https://api-docs.deepseek.com/quick_start/agent_integrations/codex/

The take-home guide has the DeepSeek config and key-entry steps. Separate credit from subscription access.

API or another provider

An OpenAI API key uses API billing. DeepSeek also publishes a Codex configuration using its own key and endpoint.

Prompt: Write the Report

PROMPT

Copy ../ml-demo/metrics.json here. Use the academic-paper skill to write a short LaTeX report with a results table, then compile a one-page PDF. Use only the numbers in metrics.json.

```
npx skills add https://github.com/imbad0202/academic-research-skills --skill academic-paper  
--agent codex --copy --yes
```

The skill supplies the report structure; every number still comes from metrics.json.

Follow-Up: Improve the Discussion

Why: the first draft is a draft; one follow-up refines it.

PROMPT

Rewrite the discussion to say that this is a single split with no uncertainty estimate, and that the degree-9 polynomial overfits. Keep every number unchanged.

`results.tex` stays editable; the values still come from `metrics.json`.

Review the draft, then ask for the change you want.

...and scored by mean squared error (MSE) on the 20 held-out points.

Results

Model	Training MSE	Held-out MSE
Mean baseline	1.2922	1.7700
Linear	0.0289	0.1035
Degree-9 polynomial	0.0223	0.1429

Discussion

The polynomial fits the training rows best (0.0223 versus 0.0289) yet generalizes worst (0.1429 versus 0.1035): extra flexibility bought a small training gain at the cost of held-out accuracy. Because the evaluation uses a single split, these numbers do not quantify uncertainty, and the held-out set should not be used for model selection. Values were copied from `metrics.json` at four decimal places and contain no new measurements.

Prompt: Review With the PDF Skill

PROMPT

Install the pdf skill, then: \$pdf render and review the report PDF for clipped text, overlaps, and spacing. Fix layout without changing the numbers.

Evidence to look for: the skill instructions load, a renderer creates page images, and those images are inspected.

A clean compile checks syntax; looking at the rendered page checks whether people can read it.

Recurring Guidance in AGENTS.md

AGENTS.md holds stable project expectations. For the experiment folder:

```
Fit only on the training rows; never on held-out rows.  
Report held-out MSE, not training error.  
Use seed 42 for data and splits.  
Use the matplotlib skill for figures.
```

Clients load guidance by scope. This is not a promise of rereading every message, nor an access-control mechanism.

<https://developers.openai.com/codex/guides/agents-md/>

<https://opencode.ai/docs/rules/>

Recurring guidance (AGENTS.md) differs from a reusable procedure (skill) and a one-off prompt.

Permissions: Who Approves What

Codex: auto-review

Approve for me sends boundary-crossing actions to a separate reviewer agent (a small model) with a rationale.

Neither replaces a sandbox; read a command before approving it.

<https://developers.openai.com/codex/sandboxing/auto-review>

<https://opencode.ai/docs/permissions/>

Modes differ by client and version; the question is always what an action may touch.

OpenCode: allow / ask / deny

Per-tool policy in config; the plan agent is read-only and asks before shell commands.

Per-Client Quick Reference

OpenCode

```
/models pick model  
/connect add provider key  
/init write AGENTS.md  
Tab build ↔ plan  
ctrl+t or /variants reasoning
```

Codex CLI

```
/model pick model  
/skills browse, $name invoke  
$skill-installer, $skill-creator  
/approve review a denial  
/init scaffold AGENTS.md
```

Verify against your installed version; command names change.

Extensions: MCP and Beyond

MCP (Model Context Protocol) is a standard way for an application to reach external tools and data through a server.

One concrete use: connect a documentation server so the agent can look things up and return a source link. This is optional and not needed for today's tasks.

<https://developers.openai.com/learn/docs-mcp>

<https://registry.modelcontextprotocol.io/>

Check the publisher, requested permissions, and where your data goes before connecting any server.

Your Responsibility, and Questions

Using an agent for coursework is allowed. The result is still yours.

- Give the agent the folder, the outcome, and constraints; plan first for anything large.
- Inspect changed files and actual output before keeping the result.
- Save useful guidance in `AGENTS.md`, or a repeated procedure as a skill.

You are responsible for what your AI generated. Review it, verify the numbers, and be ready to explain it.

The take-home guide and demo repository are linked from the course materials.

Appendix: How the Harness Assembles Context

Each turn, the application builds the model's input from several parts:

- The system prompt and the tool definitions.
- Project guidance such as `AGENTS.md`, loaded at startup.
- Files the agent opened or that you referenced, plus prior turns.

Skills are loaded on demand: only names and descriptions until one matches. Long sessions get compacted, so older detail can fall out of context.

Implementation differs by client. Treat this as the common shape, not a wire-format guarantee.

Appendix: Instruction Scope and Precedence

Scope	Where it lives
Project	AGENTS.md in the working folder or repository root.
User	Personal skills and instructions under your home directory.
Session	What you type in the current conversation.

A nearer file usually wins, but do not rely on an instruction file as enforced access control. After editing guidance, restart or explicitly ask the client to reread it.

Codex composes scoped guidance at startup; OpenCode documents its own local and global lookup.

Appendix: MCP in Detail

MCP separates the *client* (your agent application) from a *server* that exposes tools or data.

```
codex mcp add openaiDeveloperDocs --url https://developers.openai.  
com/mcp  
codex mcp list
```

A documentation server is low risk and useful: ask the agent to find how a feature works and return the source.

Check the publisher, permissions, and data destination first; remove with `codex mcp remove`.

Appendix: What a Passing Test Does Not Prove

A green test suite shows the code ran as written, not that the experiment is valid.

- A metric can be computed on the wrong split (training rows reported as held-out).
- A single split is not an uncertainty estimate.
- If the agent writes both the implementation and its test, both can share a blind spot.

Prefer an independent check: a hand-computed value, a separate oracle, or the source CSV.

This is why we check every reported number against the input data rather than trusting a summary.